Zellic

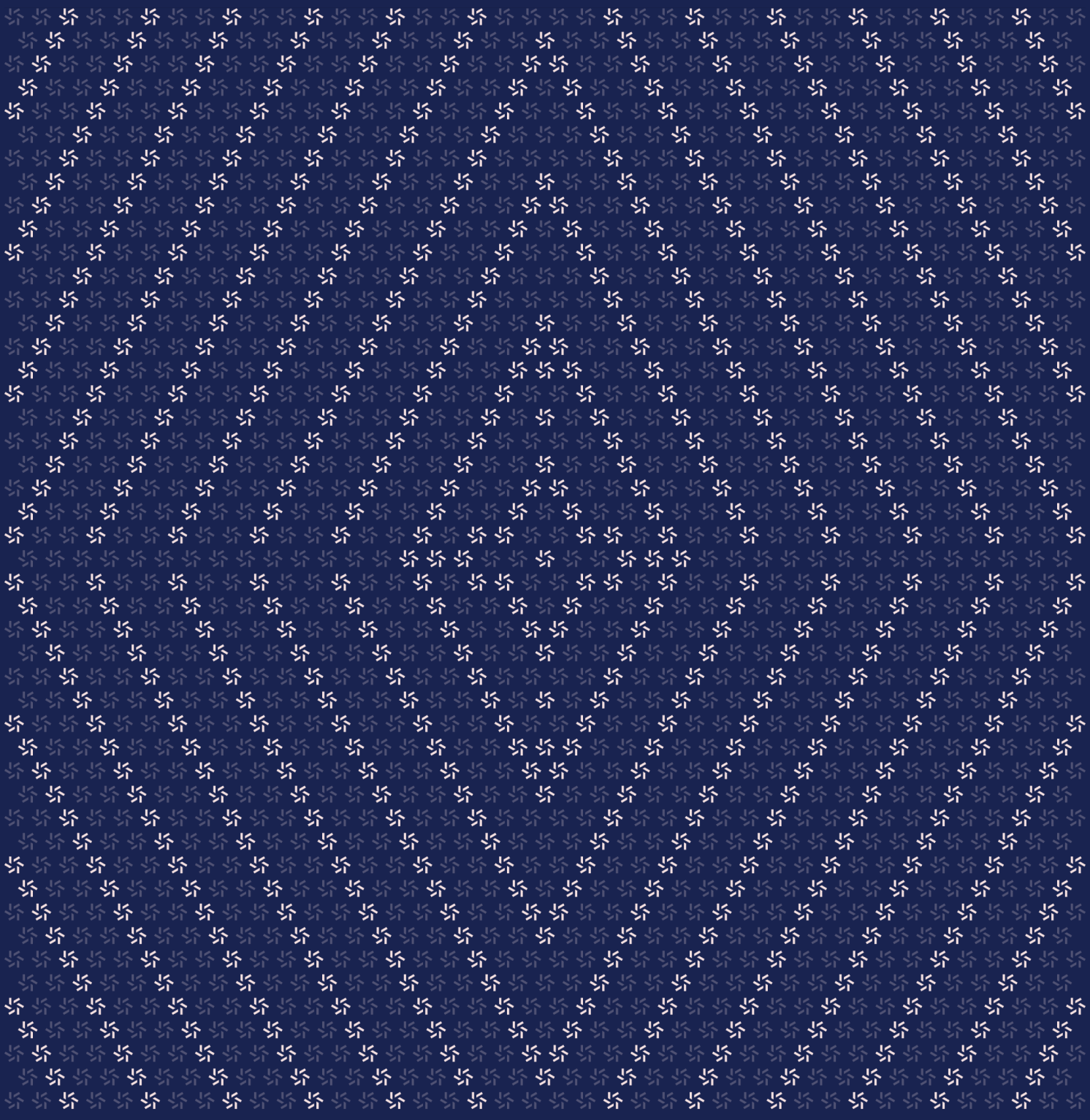
Prepared for
Julien Genestoux
Relay Protocol

Prepared by
Jisub Kim
Quentin Lemauf
Zellic

March 17, 2026

Settlement Protocol Contracts

Smart Contract Security Assessment



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Relay Protocol from March 10th to March 17th, 2026. During this engagement, Zellic reviewed the Settlement Protocol contracts' code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any ways to withdraw more funds from a depository contract than the Allocator authorized?
- Can EIP-712 signatures on the Allocator's `execute()` be replayed or forged?
- Is the ERC-6909 balance accounting on the Hub correct in all edge cases (mint, burn, `transferFrom`, operator delegation)?
- Can a single compromised oracle signer bypass the multi-sig threshold to alter Hub state?
- Are there reentrancy risks in the depository's `execute()` multicall pattern?
- Is the owner/allocator role separation on depository contracts secure — can an attacker change the allocator without going through the Security Council multi-sig?
- Are the ERC20View wrapper interactions with the ERC-6909 Hub safe from state inconsistencies?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- The Solana deposit-address program and associated Solana VM components
- The NEAR chain signatures external signing infrastructure and its integration with the Allocator
- Off-chain oracle-signing infrastructure
- MPC infrastructure
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Settlement Protocol contracts, we discovered 10 findings. One critical issue was found. One was of high impact, two were of medium impact, three were of low impact, and the remaining findings were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	1
High	1
Medium	3
Low	3
Informational	3

2. Introduction

2.1. About Settlement Protocol Contracts

Relay Protocol contributed the following description of the Settlement Protocol contracts:

> Relay Protocol is a cross-chain intent settlement protocol. Users deposit funds into Depository contracts on source chains, solvers fill user intents on destination chains, and settlement happens on the Relay chain via the Hub — an ERC-6909 ledger that tracks all account balances across all supported networks. The Oracle attests to on-chain events and updates Hub state, and the Allocator (via Near Chain Signatures) authorizes withdrawals from Depository contracts. The settlement contracts in scope include RelayHub, RelayOracle, RelayAllocator, RelayOracleMultisig, and ERC20View.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Settlement Protocol Contracts

Type	Solidity
Platform	EVM-compatible
Target	settlement-protocol
Repository	https://github.com/relayprotocol/settlement-protocol ↗
Version	`376652bcf9fe55a35719cc1d71f0543f7ad98214`

Programs	`RelayHub.sol`
	`RelayOracle.sol`
	`RelayAllocator.sol`
	`ERC20View.sol`
	`DepositAddress.sol`
	`DepositAddressFactory.sol`
	`BitcoinDepositAddress.sol`
	`BitcoinDepositSweepBuilder.sol`
	`RelayOracleMultisig.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.1 person-weeks. The assessment was conducted by two consultants over the course of 1.2 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Jisub Kim
Engineer
jisub@zellic.io ↗

Quentin Lemauf
Engineer
quentin@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 10, 2026	Kick-off call
March 10, 2026	Start of primary review period
March 17, 2026	End of primary review period

3. Detailed Findings

3.1. Permissionless sweep with unverified UTXO value burns depositor funds as miner fees

Target	BitcoinDepositAddress, BitcoinDepositSweepBuilder		
Category	Coding Mistakes	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

The `sweep()` function in BitcoinDepositAddress is permissionless and accepts caller-supplied UTXO data — including `utxo.value` — without any verification that the declared value matches the real Bitcoin UTXO. In `buildSweepPayload()`, the UTXO is decoded entirely from untrusted calldata:

```
// BitcoinDepositSweepBuilder.sol#L78-L79
function buildSweepPayload(
    bytes32 orderId,
    bytes calldata data
) external view returns (bytes memory payload, uint64 sweepAmount) {
    (UTXO memory utxo, uint64 feeRate) = abi.decode(data, (UTXO, uint64));
```

The contract then computes outputs based on the caller-supplied value and builds a single-input transaction:

```
// BitcoinDepositSweepBuilder.sol#L90-L93
sweepAmount = utxo.value - uint64(fees);
if (sweepAmount < DUST_THRESHOLD) {
    revert SweepAmountBelowDust(sweepAmount);
}
```

The sweep uses a legacy `SIGHASH_ALL` preimage that does not commit to input values — this is a property of pre-SegWit Bitcoin signing. The `buildPreImageForInput()` function serializes `[version || inputs || outputs || locktime || hashType]`, but the input value is not part of the signed data:

```
// BitcoinDepositSweepBuilder.sol#L160-L220
function buildPreImageForInput(
    BitcoinTransactionData memory txData,
    uint256 whichInput
) internal pure returns (bytes memory) {
    // ...
```

```
    // Input serialization: txid + index + scriptSig + sequence
    // NOTE: no value field in the preimage
    allInputs = bytes.concat(
        allInputs,
        prevTxidLe,
        prevIndexLe,
        scriptSigLen,
        scriptSigBytes,
        sequenceLE
    );
```

The NEAR MPC (`ChainSignatures`) is a generic ECDSA signer — it signs whatever double–SHA-256 hash the contract sends and has no Bitcoin-level awareness:

```
// BitcoinDepositAddress.sol#L202-L207
bytes memory data = ChainSignatures.encodeJSONRequest(
    ChainSignatures.stringifyBytes(abi.encodePacked(hash)),
    "Ecdsa",
    path,
    "0"
);
```

An attacker provides the correct `txid` / `index` (pointing to a real, publicly visible UTXO) but underdeclares the value. The contract builds tiny outputs, the MPC produces a valid signature, and on Bitcoin `miner_fee = sum(inputs) - sum(outputs)` — so the difference is silently burned as miner fees. The `maxFeeRate` check (line 81) does not help because the attacker uses a normal fee rate. The exploit lies in the fake input value, not the fee rate.

1. **Setup.** A deposit UTXO of 1,000,000 Sats sits at a derived deposit address.
2. **Trigger.** An attacker calls `sweep()` with the correct `txid` / `index` but declares `utxo.value = 10,000` Sats and `feeRate = 10`. The contract computes `sweepAmount = 10,000 - (10 * 269) = 7,310` Sats.
3. **Result.** The MPC signs a valid transaction. On Bitcoin, the real input is 1,000,000 Sats, outputs total ~7,310 Sats, and ~992,690 Sats are burned as miner fees.

Impact

This has several impacts.

- Any deposit UTXO can be griefed by anyone at any time since `sweep()` is permissionless and all UTXOs are publicly visible on Bitcoin.
- The depositor loses nearly all of their BTC deposit as miner fees.
- Depending on timing in the cross-chain flow, the solver may end up holding Hub tokens backed by nothing, leaving the depository insolvent.

- An attacker who is also a miner (or colluding with one) could capture the burned fees directly.

Recommendations

The root cause has two parts: unverified input values and permissionless access.

As a minimal fix, restrict `sweep()` to authorized callers (e.g., the solver or a trusted relayer) so an arbitrary attacker cannot submit a forged UTXO value. This reduces the attack surface but still trusts the authorized caller to provide correct data.

For a stronger fix, migrate from legacy P2PKH to SegWit (P2WPKH), whose BIP-143 sighash commits to input values — a mismatch between the declared and real UTXO value would produce a different hash, and Bitcoin nodes would reject the signature. This is a nontrivial change; it requires new address derivation (Bech32), a different transaction serialization format (witness fields), and reworking the payload builder. The NEAR MPC is a generic ECDSA signer and would sign a BIP-143 hash without issue, so the change is feasible but involves significant rework of the sweep builder and address generation.

Remediation

This issue has been acknowledged by Relay Protocol, and a fix was implemented in commit `4beb1a4`↗.

3.2. Multi-UTXO withdrawal charges the spender once per hash index

Target	RelayAllocator		
Category	Coding Mistakes	**Severity**	High
Likelihood	High	**Impact**	High

Description

Each call to `signWithdrawPayloadHash()` invokes `verifyWithdrawal()`, which unconditionally executes `RelayHub.transferFrom(spender, allocator, tokenId, amount)`:

```
// RelayAllocator.sol#L443-L487
function signWithdrawPayloadHash(
    SubmitWithdrawRequest calldata params,
    bytes memory signature,
    GasSettings memory gasSettings,
    uint32 hashIndex
) public {
    // ...
    // verify the withdrawal can be achieved
    verifyWithdrawal(params, payloadBuilder, signature);

    // get the hash to sign
    bytes32 hashToSign = payloadBuilder.hashToSign(
        params.chainId,
        params.depository,
        payload,
        hashIndex
    );

    _signUsingChainSignatures(
        withdrawRequestHash,
        hashToSign,
        payloadBuilder,
        gasSettings
    );
}
```

```
// RelayAllocator.sol#L606-L648
function verifyWithdrawal( ... ) internal {
    if (hasRole(APPROVED_WITHDRAWER_ROLE, msg.sender)) {
```

```
            return;
        }
        // ...
        RelayHub(hub).transferFrom(
            params.spender,
            address(this),
            tokenId,
            params.amount
        );
    }
```

For Bitcoin withdrawals built from multiple UTXOs, a separate signature is needed for each input. The caller must invoke `signWithdrawPayloadHash()` once per `hashIndex` (0, 1, 2, ...). Each `hashIndex` produces a different `hashToSign`, so the dedupe check in `_signUsingChainSignatures()` — keyed on `[withdrawRequestHash][hashToSign]` — passes every time and a fresh `transferFrom()` fires on each call.

There is no guard that tracks whether the transfer for a given `withdrawRequestHash` has already been performed.

Impact

A solver withdrawing 1 BTC through a three-input transaction is charged 3 BTC instead of 1 BTC. If the solver has insufficient balance, the second call reverts and the transaction is stuck with only one of three signatures — the BTC never arrives and the first transfer is not refunded.

If the solver has excess balance, all calls succeed, but the extra Hub tokens end up locked in the Allocator contract while the real funds backing them remain in the depository — with no refund path for either.

No attacker is needed. Any solver using the normal multi-UTXO withdrawal flow through BitcoinPayloadBuilder hits this bug by default.

Recommendations

Track whether the Hub transfer for a given `withdrawRequestHash` has already been performed. Move the `verifyWithdrawal()` call (and its `transferFrom()`) to `_submitWithdrawRequest()` so it executes exactly once when the payload is created, or add a boolean flag (e.g., `withdrawalTransferred[withdrawRequestHash]`) that `verifyWithdrawal()` checks and sets on the first call, skipping the transfer on subsequent `hashIndex` calls.

Remediation

This issue has been acknowledged by Relay Protocol, and they have provided the following response:

regarding the RelayAllocator issue you identified, this is only used for the `BitcoinPayload-Builder` - for Bitcoin withdrawals with multi-input transactions — which is precisely the use case hashIndex was designed for (signing each UTXO input separately). All the other payload builders actually ignore the hashIndex.

At the moment, I don't think we actually use the btc payload builder in production as we found it was too expensive to use the allocator to sign a payload built from multiple UTXOs.

3.3. Caller-controlled NEAR gas creates stale pending state and enables front-run denial of service

Target	BitcoinDepositAddress, RelayAllocator		
Category	Business Logic	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

Both `BitcoinDepositAddress._requestSignature()` and `RelayAllocator._signUsingChainSignatures()` let the caller choose `gasSettings.signGas` and `gasSettings.callbackGas` without any minimum bound. The contract writes `pendingSignatures` before the asynchronous NEAR MPC call resolves:

```
pendingSignatures[orderId][hash] =
    block.timestamp + PENDING_SIGNATURE_COOLDOWN;

// ...
PromiseCreateArgs memory callSign = near.call(
    nearSigner,
    "sign",
    data,
    1,
    gasSettings.signGas       // caller-controlled
);
PromiseCreateArgs memory callback = near.auroraCall(
    address(this),
    abi.encodeWithSelector(this.sweepCallback.selector, orderId, hash),
    0,
    gasSettings.callbackGas   // caller-controlled
);
callSign.then(callback).transact();
```

The same pattern exists in `RelayAllocator._signUsingChainSignatures()`.

If the caller supplies insufficient gas, the NEAR sign receipt fails or the callback never runs. When the callback does execute but sees a failed promise, it reverts instead of clearing the pending state:

```
// BitcoinDepositAddress.sol
PromiseResult memory result = AuroraSdk.promiseResult(0);
if (result.status != PromiseResultStatus.Successful) {
```

```
    revert SignCallbackFailed(orderId);
}
```

```
// RelayAllocator.sol
PromiseResult memory result = AuroraSdk.promiseResult(0);
if (result.status != PromiseResultStatus.Successful) {
    revert SignCallbackFailed(withdrawRequestHash);
}
```

The revert does not clear `pendingSignatures`, leaving it in a stale state that blocks legitimate requests for the full cooldown period (five minutes for sweep, 60 minutes for allocator withdrawals).

Because `sweep()` is permissionless and the pending key is `(orderId, hash)` — not caller-specific — an attacker can front-run a legitimate sweep request with the same calldata but deliberately low `gasSettings`. The `pendingSignatures` slot is set, the NEAR call fails, and the legitimate caller's subsequent request reverts with `SignaturePending`.

The same front-running applies to `RelayAllocator.signWithdrawPayloadHash()` through two replayable paths. The receiver-signature branch in `verifyWithdrawal()` and the `RelayAllocatorSpender.signWithdrawPayloadHash()` path both authenticate `params` but not `gasSettings`, so a third party can replay the same authorized request with lower gas and land first:

```
// RelayAllocatorSpender.sol
ALLOCATOR.signWithdrawPayloadHash(params, "", gasSettings, hashIndex);
```

The attacker's cost is only Aurora transaction gas, plus `signatureFee` in the RelayAllocator flow if nonzero.

Impact

An attacker can cause a denial of service to any sweep or withdrawal signing for the duration of the cooldown (five or 60 minutes) at negligible cost.

Repeated front-runs can keep a legitimate request permanently blocked.

Recommendations

Remove caller control over NEAR gas and use protocol-configured minimums instead, or enforce a lower bound on `signGas` and `callbackGas`. In the callbacks, when the base promise fails, clear `pendingSignatures` and return instead of reverting so the slot does not remain stale.

Remediation

This issue has been partially fixed in commit 4beb1a4 ↗ by Relay Protocol, and they have provided the following response:

> Gas minimums (MIN_SIGN_GAS = 30 TGas, MIN_CALLBACK_GAS = 20 TGas) added to BitcoinDepositAddress.sol only. RelayAllocator.sol does not yet have gas floor checks — this will be addressed in the Allocator V2 redesign (PR #339). Callback failure paths still revert instead of clearing pendingSignatures in both contracts.

3.4. Permissionless `withdrawToNear` drains the allocator's Aurora-side wNEAR operational buffer

Target	RelayAllocator		
Category	Business Logic	**Severity**	Medium
Likelihood	High	**Impact**	Medium

Description

`RelayAllocator.withdrawToNear()` is an unauthenticated external function that burns the allocator's wNEAR on Aurora and bridges it to the allocator's own NEAR representative account:

```
// RelayAllocator.sol#L491-L514
function withdrawToNear(uint256 amount) external {
    IEvmERC20(address(near.wNEAR)).withdrawToNear(
        bytes(AuroraSdk.nearRepresentative(address(this))),
        amount
    );

    PromiseCreateArgs memory unwrapCall = near.call(
        "wrap.testnet",
        "near_withdraw",
        abi.encodePacked('{"amount": "', Strings.toString(amount), '"}'),
        1,
        NEAR_WITHDRAW_GAS
    );

    unwrapCall.transact();
}
```

The destination is fixed to `AuroraSdk.nearRepresentative(address(this))`, so an attacker cannot redirect the funds to themselves. However, this still lets any caller remove the allocator's entire Aurora-side wNEAR balance at any time.

That Aurora-side wNEAR is not idle. The allocator uses it to pay for NEAR/XCC operations. In particular, `_signUsingChainSignatures()` submits a NEAR signing request with an attached balance:

```
// RelayAllocator.sol#L549-L557
PromiseCreateArgs memory callSign = near.call(
    nearSigner,
```

```
    "sign",
    data,
    1, // attachedNear
    gasSettings.signGas
);
```

The SDK explicitly documents that this attached NEAR is charged in wNEAR:

```
// aurora-xcc/Types.sol#L12-L14
/// Amount of NEAR tokens to attach to the call. This will
/// be charged from the caller in wNEAR.
uint128 nearBalance;
```

The allocator can also accumulate meaningful wNEAR over time. `init()` transfers in 2 wNEAR to bootstrap the XCC sub-account, and every signed withdrawal can collect `signatureFee` into the allocator:

```
// RelayAllocator.sol#L272-L276
near.wNEAR.transferFrom(
    msg.sender,
    address(this),
    uint256(2_000_000_000_000_000_000_000_000)
);
```

```
// RelayAllocator.sol#L449-L452
if (signatureFee > 0) {
    near.wNEAR.transferFrom(msg.sender, address(this), signatureFee);
}
```

As a result, any address can permissionlessly sweep the allocator's operational wNEAR buffer off Aurora and into the contract's NEAR-side account.

Impact

Any caller can grief the protocol by draining the allocator's Aurora-side wNEAR reserves and fee buffer.

Subsequent signing or maintenance operations that rely on allocator-held wNEAR for attached NEAR will fail or require manual operator intervention to restore the balance.

Recommendations

Restrict `withdrawToNear()` to an operator controlled role.

Remediation

This issue has been acknowledged by Relay Protocol, and a fix was implemented in commit 7073768 ↗.

3.5. Array length mismatch in `executeMultiple()` bypasses try-catch

Target	RelayOracle		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `executeMultiple()` function loops over `executions` and uses try-catch so that a failing execution does not revert the entire batch. However, `signatures[i]` is evaluated before the external self-call is entered:

```
// RelayOracle.sol#L81-L103
function executeMultiple(
    Execution[] calldata executions,
    address oracle,
    bytes[] calldata signatures
) external {
    uint256 executionsLength = executions.length;
    for (uint256 i; i < executionsLength; i++) {
        if (isExecuted[executions[i].idempotencyKey]) {
            continue;
        }

        try this.execute(executions[i], oracle, signatures[i]) {
            // Execution succeeded
        } catch {
            emit ExecutionFailed(
                executions[i].idempotencyKey,
                executions[i].actions
            );
        }
    }
}
```

If `signatures.length < executions.length`, the out-of-bounds access on `signatures[i]` reverts during argument evaluation — before `this.execute(...)` is called. The `catch` block is never reached, so the entire batch reverts and all remaining valid executions are dropped.

Impact

A mismatched `signatures` array causes the whole `executeMultiple()` call to revert, defeating its best-effort design. In practice, the caller is the oracle infrastructure, so a length mismatch is unlikely but not validated.

Recommendations

Add a length check at the top of `executeMultiple()` — `require(signatures.length == executions.length)`. This makes the precondition explicit and fails fast with a clear error rather than an opaque out-of-bounds revert mid-loop.

Remediation

This issue has been acknowledged by Relay Protocol.

3.6. Permissionless sweep allows fee maximization up to `maxFeeRate`

Target	BitcoinDepositSweepBuilder		
Category	Business Logic	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The permissionless `sweep()` flow lets any caller choose a `feeRate` up to the protocol-configured `maxFeeRate`:

```
// BitcoinDepositSweepBuilder.sol#L81-L90
if (feeRate > maxFeeRate) {
    revert FeeRateTooHigh(feeRate, maxFeeRate);
}

uint256 fees = uint256(feeRate) * SWEEP_TX_SIZE;
// ...
sweepAmount = utxo.value - uint64(fees);
```

A griefer can call `sweep()` with `feeRate = maxFeeRate`, producing a transaction with the highest possible miner fee. Since the deposit UTXO is single-spend, a legitimate sweep with a lower fee rate would be outbid on Bitcoin and fail to confirm. The depository still receives funds, but `sweepAmount` is reduced by `(maxFeeRate - reasonableFeeRate) * 269` Sats compared to what a cost-efficient sweep would deliver.

Impact

The loss per sweep is bounded by the `maxFeeRate` configuration. For example, with `maxFeeRate = 100` Sat/byte and a reasonable rate of 10 Sat/byte, the excess fee is `90 * 269 = 24,210` Sats (~$24 at $100k/BTC). The attacker does not profit unless they are also the miner. This is a known trade-off of the permissionless sweep design, and `maxFeeRate` exists as the protocol's mitigation for this scenario.

Recommendations

Consider whether `maxFeeRate` is set tightly enough relative to expected network conditions. If tighter control is desired, restrict `sweep()` to authorized callers so that fee-rate selection is not adversarial.

Remediation

This issue has been acknowledged by Relay Protocol.

3.7. The `ERC20View.transferFrom` function relies on RelayHub reverting on failed transfers

Target	ERC20View, RelayHub		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The function `ERC20View.transferFrom()` reduces allowance before attempting the Hub transfer:

```
// ERC20View.sol#L201-L223
function transferFrom(
    address from,
    address to,
    uint256 value
) public returns (bool) {
    if (msg.sender != from) {
        uint256 currentAllowance = hub.allowance(from, msg.sender, tokenId);
        // ...
        hub.approveFor(from, msg.sender, tokenId, currentAllowance - value);
    }

    bool success = hub.transferFrom(from, to, tokenId, value);
    if (success) {
        emit Transfer(from, to, value);
    }
    return success;
}
```

As of the time of writing, this is not an exploitable issue. The current `RelayHub.transferFrom()` does not fail by returning `false`. If the transfer is invalid, it reverts, and if the transfer succeeds, it returns `true`:

```
// RelayHub.sol#L249-L272
function transferFrom(
    address sender,
    address receiver,
    uint256 id,
    uint256 amount
) public returns (bool result) {
```

```
    // ...
    balanceOf[sender][id] -= amount;
    balanceOf[receiver][id] += amount;
    // ...
    return true;
}
```

If the transfer reverts, the whole call is undone, including the earlier `hub.approveFor()` change. So currently, a failed transfer cannot reduce allowance without moving funds.

This is only safe because `RelayHub.transferFrom()` reverts on failure. If a future version returns `false` instead of reverting, `ERC20View.transferFrom()` would keep the reduced allowance even though no transfer happened. In that case, the original issue would become real.

Impact

There is no issue at the time of writing. The risk is in future code changes: `ERC20View.transferFrom()` only stays safe if `RelayHub.transferFrom()` always reverts on failure. If that changes, failed transfers could start reducing allowance.

Recommendations

Document that `RelayHub.transferFrom()` must revert on failure when called by `ERC20View`. If the Hub ever returns `false` instead, update `ERC20View.transferFrom()` so allowance is only reduced after a successful transfer.

Remediation

This issue has been acknowledged by Relay Protocol, and a fix was implemented in PR #355 ↗. They intend to merge these changes into the production branch.

3.8. Single-step ownership transfer with no `new_owner` validation

Target	deposit_address (Solana)		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `set_owner` instruction transfers ownership of the deposit-address program in a single step and performs no validation on the `new_owner` parameter:

```
// deposit-address/src/lib.rs#L85-L101
pub fn set_owner(ctx: Context<SetOwner>, new_owner: Pubkey) -> Result<()> {
    let config = &mut ctx.accounts.config;
    require_keys_eq!(
        ctx.accounts.owner.key(),
        config.owner,
        DepositAddressError::Unauthorized
    );
    let previous_owner = config.owner;
    config.owner = new_owner;

    emit!(SetOwnerEvent {
        previous_owner,
        new_owner,
    });

    Ok(())
}
```

If the caller passes an incorrect pubkey (e.g., a typo, `Pubkey::default()`, or a key for which no one holds the private key), ownership is irrevocably transferred. Because the `owner` controls all admin operations — `set_depository`, `add_allowed_program`, `remove_allowed_program`, and `execute` — losing the owner key renders the entire program inoperable.

There is no zero-address check, no two-step accept pattern, and no recovery mechanism. The transfer is immediate and final.

Impact

A mistyped or default pubkey permanently locks out all administrative control. No further configuration changes, whitelist updates, or `execute` CPIs can be performed. The program continues to function for `sweep`, but operational flexibility is permanently lost.

Recommendations

Implement a two-step ownership transfer: `set_owner` stores a `pending_owner`, and a separate `accept_owner` instruction (signed by the pending owner) completes the transfer. At minimum, add a check that `new_owner != Pubkey::default()`.

Remediation

This issue has been acknowledged by Relay Protocol.

3.9. Token sweep fails when vault ATA does not exist and deposit address PDA has no SOL

Target	deposit_address (Solana)		
Category	Business Logic	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `sweep` instruction for SPL tokens invokes `relay_depository::deposit_token` via CPI. When the vault's associated token account (ATA, `vault_token_account`) does not yet exist, the depository program creates it dynamically. The payer for this account creation is the `sender` — the `deposit_address` PDA:

```
// deposit-address/src/lib.rs#L262-L284
relay_depository::cpi::deposit_token(
    CpiContext::new_with_signer(
        ctx.accounts.relay_depository_program.to_account_info(),
        relay_depository::cpi::accounts::DepositToken {
            relay_depository: ctx.accounts.relay_depository.to_account_info(),
            sender: ctx.accounts.deposit_address.to_account_info(),
            depositor: ctx.accounts.depositor.to_account_info(),
            vault: ctx.accounts.vault.to_account_info(),
            mint: mint_account.to_account_info(),
            sender_token_account:
    deposit_address_token_account.to_account_info(),
            vault_token_account: vault_token_account.to_account_info(),
            token_program: ctx.accounts.token_program.to_account_info(),
            associated_token_program: ctx
                .accounts
                .associated_token_program
                .to_account_info(),
            system_program: ctx.accounts.system_program.to_account_info(),
        },
        seeds,
    ),
    amount,
    id,
)?;
```

The `deposit_address` PDA is derived purely from the order ID and depositor and only holds SPL

tokens — it may have zero native SOL lamports (beyond the minimum rent-exempt balance of its own account, if any). If the PDA lacks sufficient SOL to pay rent for the new ATA, the CPI call fails and the sweep reverts.

This creates a scenario where tokens are deposited into the PDA's token account, but they cannot be swept until someone manually funds the PDA with native SOL.

Impact

There are two main impacts.

- Token sweeps are blocked for deposit addresses that lack native SOL, requiring manual intervention.
- Deposited tokens are stuck in the PDA until someone sends SOL to it, adding operational overhead.

Recommendations

Use a separate funded account (e.g., the transaction fee payer or a protocol-controlled account) as the payer for ATA creation instead of the deposit address PDA. Alternatively, ensure the vault ATA is precreated during configuration so the dynamic creation path is never needed.

Remediation

This issue has been acknowledged by Relay Protocol.

3.10. Owner-controlled execute whitelist provides no additional security boundary

Target	deposit_address (Solana)		
Category	Business Logic	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The `execute` instruction restricts CPI targets to programs registered in an on-chain whitelist via `AllowedProgram` PDAs:

```
// deposit-address/src/lib.rs#L641-L646
/// Validates target_program is in the whitelist
#[account(
    seeds = [ALLOWED_PROGRAM_SEED, target_program.key().as_ref()],
    bump,
    constraint = allowed_program.program_id == target_program.key(),
)]
pub allowed_program: Account<'info, AllowedProgram>,
```

However, the same `owner` that calls `execute` is also the sole authority that manages the whitelist via `add_allowed_program` and `remove_allowed_program`:

```
// deposit-address/src/lib.rs#L152-L167
pub fn add_allowed_program(ctx: Context<AddAllowedProgram>) -> Result<()> {
    require_keys_eq!(
        ctx.accounts.owner.key(),
        ctx.accounts.config.owner,
        DepositAddressError::Unauthorized
    );
    // ...
}
```

The `execute` instruction is already restricted to the `owner` via `require_keys_eq!`. The whitelist therefore creates a contradictory security model:

- If the `owner` is trusted, the whitelist is unnecessary — a trusted owner will not CPI into a malicious program.
- If the `owner` is untrusted, the whitelist is ineffective — the same `owner` can add any

program to the whitelist before calling `execute`.

In either case, the whitelist does not provide an independent security boundary. It collapses into an operational safety net against accidental misuse (e.g., calling the wrong program by mistake), but it cannot enforce any restriction that the `owner` cannot bypass on their own.

Impact

The whitelist does not constrain the `owner` in any meaningful way, since the `owner` controls both the whitelist and the `execute` instruction. It serves only as a guard against accidental miscalls during normal operation, not as a true access-control layer.

Recommendations

Clarify the intended purpose of the whitelist. If it is meant only as an operational safety net against accidental misuse, document this explicitly so future developers and auditors understand the design intent. If the whitelist is intended to act as an independent security boundary, separate its management authority from the `owner` — for example, use a multi-sig or a distinct admin role that cannot also call `execute`.

Remediation

This issue has been acknowledged by Relay Protocol.

3.11. Insufficient test coverage for Bitcoin-specific and multi-input flows

Target	Project-wide		
Category	Code Maturity	**Severity**	Medium
Likelihood	N/A	**Impact**	Medium

Description

The project has a reasonable test suite with ~60 test files and ~200 test cases covering most core contracts (Hub, Oracle, Allocator, AllocatorSpender, DepositAddressFactory, ERC20View, RelayOracleMultisig). The tests include both positive and negative scenarios for many functions.

However, coverage gaps exist in areas directly related to findings in this report:

- **The multi-UTXO withdrawal flow is untested.** No test calls `signWithdrawPayloadHash()` with `hashIndex > 0`. The only references to `hashIndex` in the test suite are in Hyperliquid payload builder tests, hardcoded to `0` with comments noting it is unused. The double-charge bug in Finding 3.2. ↗ is a direct consequence of this gap.
- **BitcoinPayloadBuilder tests are empty.** There are zero lines and zero tests. The Bitcoin-specific payload construction and fee logic are not exercised.
- **No adversarial UTXO value tests for sweep.** The DepositAddressFactory/sweep.ts tests cover the happy path and some edge cases but do not test what happens when a caller supplies a fake `utxo.value` that differs from reality — the root cause of Finding 3.1. ↗.
- **Callback failure paths are untested.** No test verifies behavior when the NEAR MPC callback returns a nonsuccessful status, leaving the stale pending state from Finding 3.3. ↗ uncovered.

Impact

The existing test suite covers most components at a surface level, which demonstrates reasonable code maturity. However, the missing negative tests and untested Bitcoin multi-input flows allowed several findings in this report to reach the audit stage. Expanding coverage to include adversarial inputs, multi-UTXO signing sequences, and asynchronous callback failure paths would catch these classes of bugs earlier.

Recommendations

We recommend expanding the test suite to cover the identified gaps: multi-UTXO withdrawal with `hashIndex > 0`, adversarial UTXO values in sweep, callback failure handling, and the

BitcoinPayloadBuilder contract. Adding integration tests that exercise the full cross-chain withdrawal and sweep flows end to end would further strengthen confidence in correctness.

Remediation

This issue has been acknowledged by Relay Protocol, and a fix was implemented in commit `4beb1a4` ↗.

4. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

4.1. Module: BitcoinDepositAddress.sol

Function: `sweep(bytes32 orderId, bytes calldata data, GasSettings calldata gasSettings)`

This function permissionlessly builds a Bitcoin sweep transaction payload via the sweep builder, stores it, and requests an MPC signature from the NEAR chain signatures protocol.

Current tests (at the time of writing) cover the sweep builder / view logic only; `BitcoinDepositAddress.sweep()` itself is not directly exercised in unit tests.

Inputs

- `orderId`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Identifies which Bitcoin deposit address to sweep — payload stored under this key.
- `data`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must be valid ABI-encoded parameters for the sweep builder (UTXO and fee rate) — the builder validates the fee rate and dust threshold.
  - **Impact**: Bitcoin transaction parameters (UTXO to spend, fee rate).
- `gasSettings`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Gas for the NEAR signing call and callback.

Branches and code coverage

Intended branches

- Payload is built via sweep builder and stored in `sweepPayloads[orderId][hash]`.

- ☐ Test coverage
- The `SweepSubmitted` event is emitted with payload and sweep amount.
  - ☐ Test coverage
- MPC signature is requested via `_requestSignature`.
  - ☐ Test coverage

Negative behavior

- Reverts if signature is already obtained for this `orderId` / hash.
  - ☐ Negative test
- Reverts if a pending signature request has not expired.
  - ☐ Negative test
- Reverts if the sweep builder reverts (e.g., fee rate too high, insufficient UTXO value, dust threshold).
  - ☐ Negative test

Function call analysis

- `sweep -> sweepBuilder.buildSweepPayload(orderId, data)`
  - **What is controllable?** `orderId` and `data`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns `(payload, sweepAmount)`. The payload is stored and later signed. A compromised sweep builder could return a payload that sends funds to an attacker-controlled address. The sweep builder is set by the owner.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `sweep -> sweepBuilder.hashToSign(payload)`
  - **What is controllable?** `payload` from the previous call.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the hash to be signed by MPC — used as a key for storing the payload and requesting the signature.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A. Pure function.
- `sweep -> _requestSignature(orderId, hash, gasSettings)`
  - **What is controllable?** `orderId`, `hash`, and `gasSettings`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** Asynchronous NEAR request — completion is handled later by

`sweepCallback`.

Function: `sweepCallback(bytes32 orderId, bytes32 _hashToSign)`

This function is a callback invoked by the NEAR cross-contract call system after the MPC signer processes a signing request. It stores the resulting signature and emits a `SweepSigned` event. It is only callable by the Aurora SDK's NEAR representative implicit address.

Inputs

- `orderId`
  - **Control**: Controllable by the NEAR MPC signer via Aurora XCC callback.
  - **Constraints**: Not checked against `pendingSignatures` or stored payload state inside this function.
  - **Impact**: Identifies which order's signature is being stored.
- `_hashToSign`
  - **Control**: Controllable by the NEAR MPC signer via Aurora XCC callback.
  - **Constraints**: Not validated against a previously requested hash inside this function.
  - **Impact**: Key to store the signed payload.

Branches and code coverage

Intended branches

- Successful signing — signature stored in `signedPayloads[orderId][_hashToSign]` and `SweepSigned` emitted.
  - ☐ Test coverage (requires Aurora precompiles)

Negative behavior

- Reverts if the caller is not the NEAR representative implicit address.
  - ☐ Negative test (requires Aurora precompiles)
- Reverts if the NEAR promise result status is not `Successful`.
  - ☐ Negative test (requires Aurora precompiles)

4.2. Module: DepositAddress.sol

Function: `sweep(address token, address depositor, bytes32 orderId)`

This function sweeps the entire balance of a token (ERC-20 or native ETH) from this deposit address to the `RelayDepository`. It is only callable by the factory. For native ETH, it calls `depositNative`; for ERC-20 tokens, it approves and calls `depositErc20`.

Inputs

- `token`
  - **Control**: Controllable by the factory.
  - **Constraints**: While `address(0)` indicates native ETH, any other address is treated as ERC-20.
  - **Impact**: Determines sweep path (native vs ERC-20).
- `depositor`
  - **Control**: Controllable by the factory.
  - **Constraints**: None.
  - **Impact**: Address credited in the depository.
- `orderId`
  - **Control**: Controllable by the factory.
  - **Constraints**: None.
  - **Impact**: Deposit identifier passed to the depository.

Branches and code coverage

Intended branches

- Native ETH: if `balance > 0`, it calls `DEPOSITORY.depositNative{value: balance}(depositor, orderId)`.
  - ☑ Test coverage
- ERC-20: if `balance > 0`, it approves the depository then calls `DEPOSITORY.depositErc20(depositor, token, balance, orderId)`.
  - ☑ Test coverage
- Zero balance.
  - ☑ Test coverage

Negative behavior

- Unauthorized direct calls are tested as generic rejections; the exact `OnlyFactory` custom error is not asserted.

- ☐ Negative test
- Potential revert if the ERC-20 token's `approve` or `balanceOf` fails (nonstandard token).
  - ☐ Negative test

4.3. Module: DepositAddressFactory.sol

Function: `sweep(bytes32 orderId, address depositor, address[] calldata tokens)`

This function permissionlessly deploys a deterministic EIP-1167 minimal proxy (if not already deployed) and sweeps all specified tokens from it to the depository. The proxy address is computed from `keccak256(orderId, depositor)` as the `CREATE2` salt.

Inputs

- `orderId`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Identifies the deposit order — determines proxy address via `CREATE2` salt.
- `depositor`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Address credited in the depository — part of `CREATE2` salt.
- `tokens`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: List of tokens swept from the proxy — each swept independently.

Branches and code coverage

Intended branches

- Proxy is not yet deployed — deploys via `LibClone.cloneDeterministic` and emits `ProxyDeployed`.
  - ☑ Test coverage
- Proxy is already deployed.
  - ☑ Test coverage
- ERC-20 tokens are swept to the depository.

☑ Test coverage

- Native ETH is swept to the depository.

☑ Test coverage

- Multiple tokens are swept in a single call.

☑ Test coverage

- Correct depositor attribution in the depository.

☑ Test coverage

Negative behavior

- Incorrect depositor computes different proxy address — funds at the wrong address are not swept.

☑ Negative test

- No validation that the `tokens` array contains no duplicates (duplicate sweeps succeed but second is zero-balance).

☑ Negative test

Function call analysis

- `sweep -> LibClone.predictDeterministicAddress(IMPLEMENTATION, salt, address(this))`
  - **What is controllable?** `salt` derived from `orderId` and `depositor`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the deterministic address. Used to check if proxy exists and to target the sweep.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A. Pure computation.
- `sweep -> LibClone.cloneDeterministic(IMPLEMENTATION, salt)`
  - **What is controllable?** `salt` from caller inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the deployed proxy address used as the sweep target.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `sweep -> _sweepProxy -> DepositAddress(proxy).sweep(tokens[i], depositor, orderId)` (per token)
  - **What is controllable?** `tokens[i]`, `depositor`, and `orderId`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?**

N/A.

4.4. Module: RelayAllocator.sol

Function: `signWithdrawCallback(bytes32 withdrawRequestHash, bytes32 hashToSign)`

This function is a callback invoked by the NEAR cross-contract call system after the MPC signer processes a signing request. It stores the resulting signature in `signedPayloads` and emits `PayloadWithdrawSigned`. It is only callable by the Aurora SDK's NEAR representative implicit address.

Inputs

- `withdrawRequestHash`
  - **Control**: Controllable by the NEAR MPC signer via Aurora XCC callback.
  - **Constraints**: Not validated against `pendingSignatures` or prior submission inside this function.
  - **Impact**: Identifies which withdrawal request's signature is being stored.
- `hashToSign`
  - **Control**: Controllable by the NEAR MPC signer via Aurora XCC callback.
  - **Constraints**: Not checked against stored request state inside this function.
  - **Impact**: Secondary key to store the signed payload.

Branches and code coverage

Intended branches

- Successful signing — signature is stored in `signedPayloads[withdrawRequestHash][hashToSign]` and `PayloadWithdrawSigned` is emitted.
  - ☐ Test coverage (requires Aurora precompiles)

Negative behavior

- Reverts if the caller is not the NEAR representative implicit address.
  - ☐ Negative test (requires Aurora precompiles)
- Reverts if the NEAR promise result status is not `Successful`.
  - ☐ Negative test (requires Aurora precompiles)

Function: `signWithdrawPayloadHash(SubmitWithdrawRequest calldata params, bytes memory signature, GasSettings memory gasSettings, uint32 hashIndex)`

This function triggers the NEAR MPC signing of a previously submitted withdrawal payload. It verifies the delay has passed, validates the withdrawal authorization, computes the hash to sign via the payload builder, and initiates a NEAR cross-contract call to the chain signatures signer.

Inputs

- `params`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must hash to an existing withdrawal request (`payloads[hash].length > 0`) — must exactly match the original submission.
  - **Impact**: Recomputes the withdrawal request hash and verifies authorization.
- `signature`
  - **Control**: Controllable by the caller.
  - **Constraints**: Valid EIP-712 receiver signature required only when the caller is not an approved withdrawer, spender, or operator; this path is further gated by `params.spender == receiver alias` (RelayAllocator.sol:631).
  - **Impact**: Authorizes withdrawal when the caller is not directly authorized.
- `gasSettings`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Gas for the NEAR MPC signing call.
- `hashIndex`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must be a valid index for the payload builder's `hashToSign` function.
  - **Impact**: Selects which payload-builder hash is requested, but each call reruns authorization and `Hub.transferFrom` before hash selection.

Branches and code coverage

Intended branches

- The signature fee is collected if greater than zero (wNEAR `transferFrom`).
  - ☑ Test coverage
- The payload exists, and the delay has passed.
  - ☑ Test coverage

- The caller is an approved withdrawer.
  - ☑ Test coverage
- The caller is the spender.
  - ☑ Test coverage
- The caller is an operator for the spender.
  - ☑ Test coverage
- The caller provides a valid receiver signature.
  - ☐ Test coverage (success test is skipped)
- The Hub balance is transferred from the spender to the allocator.
  - ☑ Test coverage

Negative behavior

- Reverts if the payload does not exist.
  - ☑ Negative test
- Reverts if the delay has not passed.
  - ☑ Negative test
- Reverts if the caller is not authorized and no valid signature is provided.
  - ☑ Negative test
- Reverts if the hash was already signed.
  - ☐ Negative test
- Reverts if a pending signature has not expired (60-minute cooldown).
  - ☐ Negative test
- Repeated calls with the same withdrawal request but different `hashIndex` values can transfer the withdrawal amount multiple times before each signature request.
  - ☑ Negative test

Function call analysis

- `signWithdrawPayloadHash -> near.wNEAR.transferFrom(msg.sender, address(this), signatureFee)`
  - **What is controllable?** `msg.sender`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Standard ERC-20 transfer — reverts if the caller has insufficient wNEAR allowance/balance.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

- `signWithdrawPayloadHash -> verifyWithdrawal(params, payloadBuilder, signature)`
    - **What is controllable?** `params` and `signature`.
    - **If the return value is controllable, how is it used and how can it go wrong?** No return value — reverts if not authorized.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `signWithdrawPayloadHash -> IPayloadBuilder(builder).hashToSign(params.chainId, params.depository, payload, hashIndex)`
    - **What is controllable?** `hashIndex`.
    - **If the return value is controllable, how is it used and how can it go wrong?** Returns the hash to be signed by MPC. Hash selection happens after `verifyWithdrawal()` and `Hub.transferFrom()`, so repeated calls across `hashIndex` values can move funds multiple times before signature completion.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `signWithdrawPayloadHash -> _signUsingChainSignatures(withdrawRequestHash, hashToSign, payloadBuilder, gasSettings)`
    - **What is controllable?** `gasSettings`.
    - **If the return value is controllable, how is it used and how can it go wrong?** No return value — initiates an asynchronous NEAR call.
    - **What happens if it reverts, reenters or does other unusual control flow?** Asynchronous NEAR request — completion is handled later by `signWithdrawCallback`.

Function: `submitAndSignWithdrawRequest(SubmitWithdrawRequest calldata params, bytes memory signature, GasSettings memory gasSettings)`

This function combines `_submitWithdrawRequest` and `signWithdrawPayloadHash` in a single transaction. It only works for payloads requiring a single signature (`hashIndex = 0`).

Inputs

- `params`
    - **Control**: Controllable by the caller.
    - **Constraints**: Same as `submitWithdrawRequest`.
    - **Impact**: Defines the withdrawal request.
- `signature`

- **Control**: Controllable by the caller.
- **Constraints**: Valid EIP-712 receiver signature is only one authorization path and only applies when `params.spender` matches the receiver alias (RelayAllocator.sol:631); approved withdrawers and authorized spender/operator paths can also succeed without it.
- **Impact**: Authorizes withdrawal on behalf of the receiver.

- `gasSettings`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Gas for NEAR signing operations.

Branches and code coverage

Intended branches

- Submit and sign in a single transaction (delay must be zero).
  - ☑ Test coverage
- Works only when delay is zero (otherwise, `signWithdrawPayloadHash` reverts with `PayloadNotReady`).
  - ☑ Test coverage

Negative behavior

- Reverts if delay is greater than zero (timestamp check fails because `block.timestamp + delay > block.timestamp`).
  - ☑ Negative test

Function call analysis

- `submitAndSignWithdrawRequest -> _submitWithdrawRequest(params)`
  - **What is controllable?** `params`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns `withdrawRequestHash` — not used directly since `signWithdrawPayloadHash` recomputes it.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `submitAndSignWithdrawRequest -> signWithdrawPayloadHash(params, signature, gasSettings, 0)`
  - **What is controllable?** All parameters.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.

- **What happens if it reverts, reenters or does other unusual control flow?** See the threat model for `signWithdrawPayloadHash` above.

Function: `submitWithdrawRequest(SubmitWithdrawRequest calldata params)`

This function submits a withdrawal request. It delegates to an internal `_submitWithdrawRequest`, which validates the payload builder exists, builds the withdrawal payload, stores it, and sets a timestamp with delay enforcement.

Inputs

- `params.chainId`
  - **Control**: Controllable by the caller.
  - **Constraints**: A payload builder must be configured for this chain ID and depository.
  - **Impact**: Destination chain for the withdrawal.
- `params.depository`
  - **Control**: Controllable by the caller.
  - **Constraints**: A payload builder must be configured for this depository on the given chain.
  - **Impact**: Depository account on the destination chain.
- `params.currency`
  - **Control**: Controllable by the caller.
  - **Constraints**: Validated by the payload builder.
  - **Impact**: Currency to be withdrawn.
- `params.amount`
  - **Control**: Controllable by the caller.
  - **Constraints**: Validated by the payload builder (e.g., `InsufficientAmount`).
  - **Impact**: Withdrawal amount.
- `params.spender`
  - **Control**: Controllable by the caller.
  - **Constraints**: Checked during signing, not at submit time.
  - **Impact**: Hub address whose balance will be debited.
- `params.receiver`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Destination address on the target chain.
- `params.data`

- **Control**: Controllable by the caller.
- **Constraints**: Passed to the payload builder.
- **Impact**: Additional data for the payload builder.

- `params.nonce`
  - **Control**: Controllable by the caller.
  - **Constraints**: Checked for replay during signing, not at submit time.
  - **Impact**: Replay protection for signature-based withdrawals.

Branches and code coverage

Intended branches

- The payload builder exists.
  - ☑ Test coverage
- The payload timestamp set with delay (global or depository-specific).
  - ☑ Test coverage
- The `PayloadBuilt` event is emitted.
  - ☑ Test coverage
- Depository-specific delay is used when configured.
  - ☑ Test coverage
- Global delay is used as fallback.
  - ☑ Test coverage

Negative behavior

- Reverts if no builder is configured for the `chainId` / depository.
  - ☑ Negative test
- Reverts if the same request hash already has a timestamp.
  - ☑ Negative test

Function call analysis

- `submitWithdrawRequest -> _submitWithdrawRequest -> IPayloadBuilder(builder).buildPayload(...)`
  - **What is controllable?** All parameters.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the unsigned payload bytes stored in `payloads[withdrawRequestHash]`. A compromised builder could return a

malicious payload.
- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `withdrawToNear(uint256 amount)`

This function permissionlessly withdraws wNEAR from the Allocator contract to the NEAR network and unwraps it.

Inputs

- `amount`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Amount of wNEAR to withdraw and unwrap.

Branches and code coverage

Intended branches

- wNEAR is withdrawn from the contract to the NEAR network.
  - ☑ Test coverage (covered only at the Aurora-side token-movement level, not end-to-end NEAR settlement)
- wNEAR is unwrapped on the NEAR network.
  - ☐ Test coverage (no end-to-end NEAR unwrap assertion)

Negative behavior

- Reverts if the contract's wNEAR balance is insufficient.
  - ☐ Negative test (requires Aurora/NEAR integration)

Function call analysis

- `withdrawToNear -> IEvmERC20(address(near.wNEAR)).withdrawToNear(bytes(nearRepresentative), amount)`
  - **What is controllable?** `amount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Bridges wNEAR to NEAR.

    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `withdrawToNear -> near.call("wrap.testnet", "near_withdraw", ..., 1, NEAR_WITHDRAW_GAS)`
    - **What is controllable?** `amount` in the JSON payload.
    - **If the return value is controllable, how is it used and how can it go wrong?** No return value used directly — the NEAR call unwraps wNEAR.
    - **What happens if it reverts, reenters or does other unusual control flow?** The unwrap step is scheduled asynchronously; downstream NEAR-side failure does not roll back the current Aurora transaction.

4.5. Module: RelayHub.sol

Function: `approve(address spender, uint256 id, uint256 amount)`

This function sets the allowance for a spender on a specific token ID from the caller's balance. It overwrites any previous allowance.

Inputs

- `spender`
    - **Control**: Controllable by the caller.
    - **Constraints**: None.
    - **Impact**: Address authorized to spend the caller's tokens.
- `id`
    - **Control**: Controllable by the caller.
    - **Constraints**: None.
    - **Impact**: Token ID for which the allowance is set.
- `amount`
    - **Control**: Controllable by the caller.
    - **Constraints**: None.
    - **Impact**: Allowance amount set, overwriting any existing value.

Branches and code coverage

Intended branches

- Allowance is updated in `allowance[msg.sender][spender][id]`.
    - ☑ Test coverage

- The approval event is emitted.
  - ☑ Test coverage
- The ERC20View approval event is forwarded.
  - ☑ Test coverage

Negative behavior

- Reverts if no ERC20View has been created for `id`, because approval event forwarding calls `erc20Views[id]`.
  - ☐ Negative test

Function call analysis

- `approve -> _emitERC20ViewApprovalEvent(id, msg.sender, spender, amount)`
  - **What is controllable?** `id`, `spender`, and `amount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** Calls `IERC20View(erc20Views[id]).emitApprovalEvent(...)`. If no ERC20View exists, calls to zero address and reverts. No reentrancy risk from the canonical implementation.

Function: `approveFor(address owner, address spender, uint256 id, uint256 amount)`

This function sets the allowance on behalf of an owner. It is restricted to the ERC20View contract corresponding to the given token ID, enabling the ERC20View `approve` and `transferFrom` flows to update Hub-level allowances.

Inputs

- `owner`
  - **Control**: Controllable by the ERC20View caller.
  - **Constraints**: None.
  - **Impact**: Address whose allowance is set.
- `spender`
  - **Control**: Controllable by the ERC20View caller.
  - **Constraints**: None.
  - **Impact**: Address being authorized to spend.

- `id`
    - **Control**: Controllable by the ERC20View (immutable `tokenId`).
    - **Constraints**: `erc20Views[id]` must equal `msg.sender`.
    - **Impact**: Token ID for which the allowance is set.
- `amount`
    - **Control**: Controllable by the ERC20View caller.
    - **Constraints**: None.
    - **Impact**: Allowance amount set.

Branches and code coverage

Intended branches

- The caller is the ERC20View for token ID.
    - ☑ Test coverage
- The approval event is emitted on Hub.
    - ☑ Test coverage

Negative behavior

- Reverts if the caller is not the ERC20View for the token ID.
    - ☑ Negative test

Function: `permit(address owner, address spender, uint256 tokenId, uint256 value, uint256 deadline, uint8 v, bytes32 r, bytes32 s)`

This function implements gasless approval via EIP-712 signatures. It allows a third party to submit a signed permit to set an allowance on behalf of the owner without requiring the owner to send a transaction.

Inputs

- `owner`
    - **Control**: Controllable by the caller.
    - **Constraints**: Must not be `address(0)` — the recovered signer must equal `owner`.
    - **Impact**: Address whose allowance is set.
- `spender`
    - **Control**: Controllable by the caller.

- **Constraints**: Must match the value in the signature.
- **Impact**: Address authorized to spend.

- `tokenId`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must match the value in the signature.
  - **Impact**: Token ID for which the allowance is set.
- `value`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must match the value in the signature.
  - **Impact**: Allowance amount.
- `deadline`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must be greater than or equal to `block.timestamp`.
  - **Impact**: Prevents stale permits from being replayed.
- `v`, `r`, `s`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must form a valid ECDSA signature recovering to `owner`.
  - **Impact**: Authenticates the permit.

Branches and code coverage

Intended branches

- A valid permit sets allowance for (owner, spender, `tokenId`).
  - ☑ Test coverage
- The nonce is incremented after successful permit.
  - ☑ Test coverage
- The approval event is emitted.
  - ☑ Test coverage
- The ERC20View approval event is forwarded.
  - ☑ Test coverage
- A permit with `type(uint256).max` value sets unlimited allowance.
  - ☑ Test coverage
- Multiple permits from the same owner increment the nonce correctly.
  - ☑ Test coverage

Negative behavior

- Reverts if the owner is `address(0)`.
  - ☑ Negative test
- Reverts if `block.timestamp > deadline`.
  - ☑ Negative test
- Reverts if the recovered signer does not equal the owner.
  - ☑ Negative test
- Reverts on replay (nonce mismatch causes signature recovery to produce the wrong address).
  - ☑ Negative test

Function call analysis

- `permit -> ECDSA.recover(hash, v, r, s)`
  - **What is controllable?** `v`, `r`, and `s` — `hash` is deterministically computed from inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the recovered address. If it does not match `owner`, reverts. An attacker cannot forge a signature without the owner's private key.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `permit -> _emitERC20ViewApprovalEvent(tokenId, owner, spender, value)`
  - **What is controllable?** `tokenId`, `owner`, `spender`, and `value`.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** If `erc20Views[tokenId]` is `address(0)`, event forwarding reverts even if signature validation succeeded.

Function: `setOperator(address operator, bool approved)`

This function sets or unsets a per-account operator for the caller. An approved operator can call `transferFrom` on behalf of the caller without needing token-level allowance.

Inputs

- `operator`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.

- **Impact**: Operator status (address granted or revoked) over the caller's tokens.

- `approved`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: `true` grants and `false` revokes operator status.

Branches and code coverage

Intended branches

- Operator status is set in `_isOperator[msg.sender][operator]`.
  - ☐ Test coverage
- The `OperatorSet` event is emitted.
  - ☐ Test coverage

Negative behavior

- No revert conditions.

Function call analysis

- `setOperator -> _setOperatorFor(msg.sender, operator, approved)`
  - **What is controllable?** `operator` and `approved`.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `transfer(address receiver, uint256 id, uint256 amount)`

This function transfers ERC-6909 tokens from the caller to a receiver. It decrements the caller's balance and increments the receiver's balance for the given token ID. It emits both a Hub-level `Transfer` event and forwards a transfer event to the corresponding ERC20View contract.

Inputs

- `receiver`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.

    - **Impact**: Token recipient.
- `id`
    - **Control**: Controllable by the caller.
    - **Constraints**: None.
    - **Impact**: Token type transferred.
- `amount`
    - **Control**: Controllable by the caller.
    - **Constraints**: Must not exceed `balanceOf[msg.sender][id]`.
    - **Impact**: Quantity transferred.

Branches and code coverage

Intended branches

- The sender balance decreases by `amount`.
    - ☐ Test coverage
- The receiver balance increases by `amount`.
    - ☐ Test coverage
- The transfer event is emitted with correct parameters.
    - ☐ Test coverage
- The ERC20View transfer event is forwarded.
    - ☑ Test coverage

Negative behavior

- Reverts on insufficient balance.
    - ☐ Negative test

Function call analysis

- `transfer -> _emitERC20ViewEvent(id, msg.sender, receiver, amount)`
    - **What is controllable?** `id`, `receiver`, and `amount`.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** Calls `IERC20View(erc20Views[id]).emitTransferEvent(...)`. If `erc20Views[id]` is `address(0)`, event forwarding reverts even if balance updates would otherwise succeed. The canonical ERC20View only emits an event — no reentrancy risk.

Function: `transferFrom(address sender, address receiver, uint256 id, uint256 amount)`

This function transfers ERC-6909 tokens from one address to another. The caller must be the sender, an operator, or the registered ERC20View for the token ID or hold sufficient allowance. Allowance is decremented unless infinite (`type(uint256).max`).

Inputs

- `sender`
  - **Control**: Controllable by the caller.
  - **Constraints**: Caller must be authorized (sender, operator, ERC20View, or have allowance).
  - **Impact**: Address whose balance is debited.
- `receiver`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Token recipient.
- `id`
  - **Control**: Controllable by the caller.
  - **Constraints**: None.
  - **Impact**: Token type transferred — determines the ERC20View bypass check.
- `amount`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must not exceed `balanceOf[sender][id]` — if allowance-based, must not exceed `allowance[sender][msg.sender][id]`.
  - **Impact**: Quantity transferred.

Branches and code coverage

Intended branches

- The caller is the sender.
  - ☐ Test coverage
- The caller is an operator.
  - ☐ Test coverage
- The caller is the ERC20View for the token ID.
  - ☑ Test coverage (indirectly covered via `ERC20View.transfer` / `transferFrom`)
- The caller has sufficient allowance.

- ☑ Test coverage (allowance path covered directly, other branches not)

- The caller has `type(uint256).max` allowance.
  - ☑ Test coverage (allowance path covered directly, other branches not)
- Sender balance decreases; receiver balance increases.
  - ☑ Test coverage
- The transfer event is emitted.
  - ☐ Test coverage
- The ERC20View transfer event is forwarded.
  - ☑ Test coverage

Negative behavior

- Reverts on insufficient allowance.
  - ☑ Negative test
- Reverts on insufficient sender balance.
  - ☐ Negative test (not directly asserted on `RelayHub.transferFrom`)

Function call analysis

- `transferFrom -> isOperator(sender, msg.sender)`
  - **What is controllable?** `sender`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns `true` if `msg.sender` has `OPERATOR_ROLE` or is a per account operator. An attacker with operator status can transfer any of the sender's tokens without allowance.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A. Pure view call on internal state.
- `transferFrom -> _emitERC20ViewEvent(id, sender, receiver, amount)`
  - **What is controllable?** `id`, `sender`, `receiver`, and `amount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** Calls ERC20View to emit an event. If `erc20Views[id]` is `address(0)`, event forwarding reverts even if balance updates would otherwise succeed. No reentrancy risk from canonical implementation.

4.6. Module: RelayOracle.sol

Function: `execute(Execution calldata execution, address oracle, bytes calldata signature)`

This function executes a set of oracle-signed actions (mint, burn, transfer) on the RelayHub. It verifies the oracle has `ORACLE_ROLE`, validates the EIP-712 signature (supporting both ECDSA and EIP-1271 contract signatures), marks the idempotency key as executed, and iterates through the action list.

Inputs

- `execution`
  - **Control**: Controllable by the caller.
  - **Constraints**: `idempotencyKey` must not have been executed before.
  - **Impact**: Defines operations on the Hub — the idempotency key prevents replay.
- `oracle`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must have `ORACLE_ROLE`.
  - **Impact**: Identifies the signer.
- `signature`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must be a valid signature by `oracle` over the EIP-712 hash.
  - **Impact**: Authenticates the execution.

Branches and code coverage

Intended branches

- Valid oracle signature — actions are executed and idempotency key is marked.
  - ☑ Test coverage
- Mint action — `HUB.mint(hubToAddress, hubTokenId, amount)` is called.
  - ☑ Test coverage
- Burn action — `HUB.burn(hubFromAddress, hubTokenId, amount)` is called.
  - ☑ Test coverage
- Transfer action — `HUB.transferFrom(hubFromAddress, hubToAddress, hubTokenId, amount)` is called.
  - ☑ Test coverage

- Multiple actions in one execution.
  - ☑ Test coverage
- The `Executed` event is emitted with idempotency key and actions.
  - ☑ Test coverage

Negative behavior

- Reverts if the idempotency key was already used.
  - ☑ Negative test
- Reverts if oracle does not have `ORACLE_ROLE`.
  - ☑ Negative test
- Reverts if signature verification fails.
  - ☑ Negative test
- Reverts if the decoded `actionType` is outside the `ActionType` enum range.
  - ☐ Negative test

Function call analysis

- `execute -> _execute(execution, oracle, signature)`
  - **What is controllable?** All parameters.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `_execute -> SignatureChecker.isValidSignatureNow(oracle, digest, signature)`
  - **What is controllable?** `oracle` and `signature` — `digest` is deterministically computed.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns `bool` — if `false`, reverts with `InvalidSignature`. Supports ECDSA and EIP-1271.
  - **What happens if it reverts, reenters or does other unusual control flow?** For EIP-1271 contract signers, signature verification performs an external call before `isExecuted` is set, so a malicious authorized oracle contract can reenter during validation.
- `_execute -> _executeAction(action)` (per action)
  - **What is controllable?** `action`.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.

- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

- `_executeAction -> HUB.mint/burn/transferFrom`
  - **What is controllable?** Parameters decoded from the signed action data.
  - **If the return value is controllable, how is it used and how can it go wrong?** Return values are not checked (success implied by nonrevert).
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `executeMultiple(Execution[] calldata executions, address oracle, bytes[] calldata signatures)`

This function executes multiple oracle-signed executions in a single transaction.

Inputs

- `executions`
  - **Control**: Controllable by the caller.
  - **Constraints**: Each execution is validated independently inside `this.execute()`.
  - **Impact**: Defines the batch of operations to perform.
- `oracle`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must have `ORACLE_ROLE` (checked per `execute` call).
  - **Impact**: Single oracle for all executions in the batch. A single oracle parameter is shared across the entire batch.
- `signatures`
  - **Control**: Controllable by the caller.
  - **Constraints**: Each signature must be valid for the corresponding execution.
  - **Impact**: Authenticates each execution independently.

Branches and code coverage

Intended branches

- All valid executions succeed and mint/burn/transfer correctly.
  - ☑ Test coverage
- Already executed idempotency keys are skipped.
  - ☑ Test coverage

- Multiple executions with separate idempotency keys succeed.

  ☑ Test coverage

Negative behavior

- Failed execution emits the `ExecutionFailed` event and continues with next (try-catch).

  ☑ Negative test
- An already executed key in batch is silently skipped.

  ☑ Negative test
- No check that `executions.length == signatures.length` — out-of-bounds access on `signatures[i]` reverts the entire call.

  ☐ Negative test

Function call analysis

- `executeMultiple -> this.execute(executions[i], oracle, signatures[i])`
  - **What is controllable?** All parameters.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts from `this.execute(...)` are caught by try-catch, so `ExecutionFailed` is emitted and the batch continues. Errors that occur before entering the `try` block, such as `signatures[i]` out-of-bounds access, still revert the entire call.

5. Assessment Results

During our assessment on the scoped Settlement Protocol contracts, we discovered 10 findings. One critical issue was found. One was of high impact, two were of medium impact, three were of low impact, and the remaining findings were informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.